REACTJS

Prof. K. T. Mane

D Y PATIL
COLLEGE of
ENGINEERING & TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
KASABA BAWADA, KOLHAPUR



INTRODUCTION

- ReactJS is a library for web and native user interface.
- It is an open-source, component-based front-end library that is responsible only for the view layer of the application
- ReactJS is not a framework, it is just a library developed by Facebook
- It designs simple views for each state in your application, and React will efficiently update and render just the right component when your data changes.

FEATURES OF *REACT LIBRARY* –

- Solid base architecture
- Extensible architecture
- Component based library
- JSX (JavaScript XML) based design architecture
- Declarative UI library

BENEFITS

- Easy to learn
- Easy to adept in modern as well as legacy application
- Faster way to code a functionality
- Availability of large number of ready-made component
- Large and active community

APPLICATIONS

- *Facebook*, popular social media application
- *Instagram*, popular photo sharing application
- *Netflix*, popular media streaming application
- *Code Academy*, popular online training application
- *Reddit*, popular content sharing application

INSTALLATION

- For installation of ReactJS, node.js is required.
- The Node.js is downloaded from

https://nodejs.org/en/download

- Select the 18.16.0 LTS -64 bit windows installer, download it and install it

Downloads

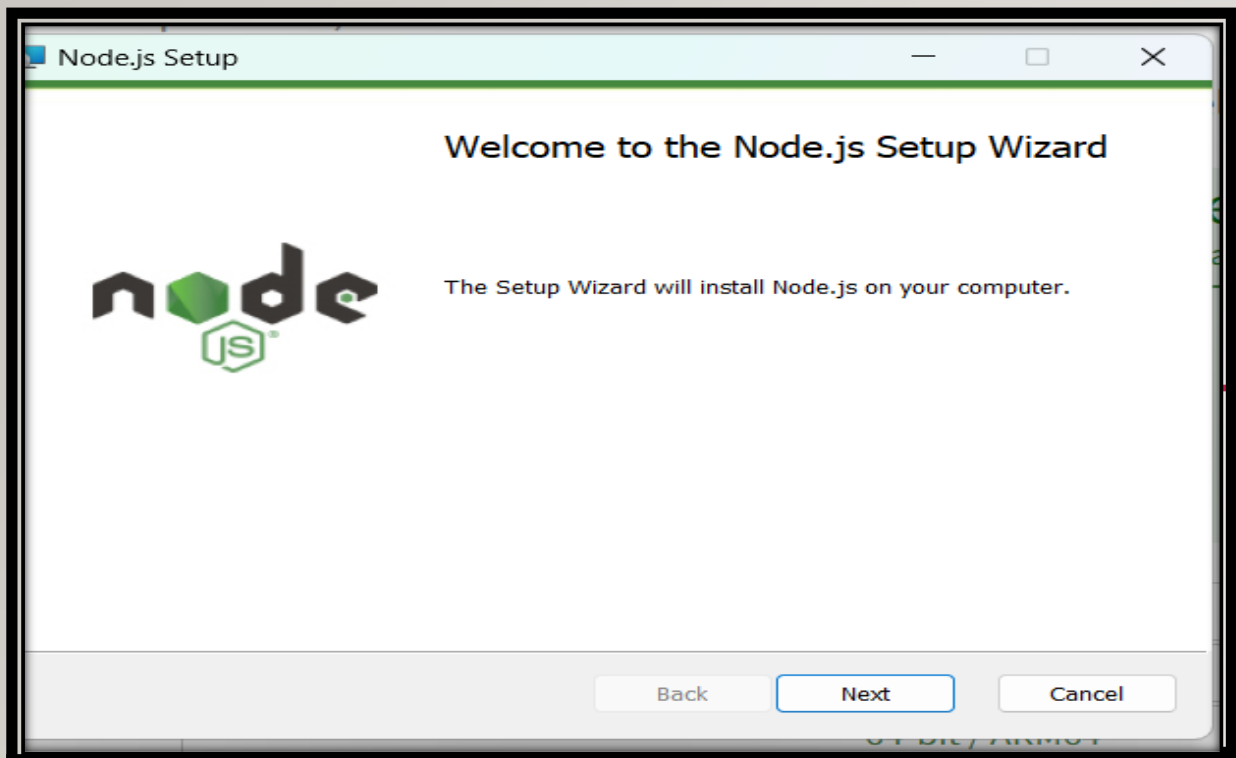
Latest LTS Version: **18.16.0** (includes npm 9.5.1)

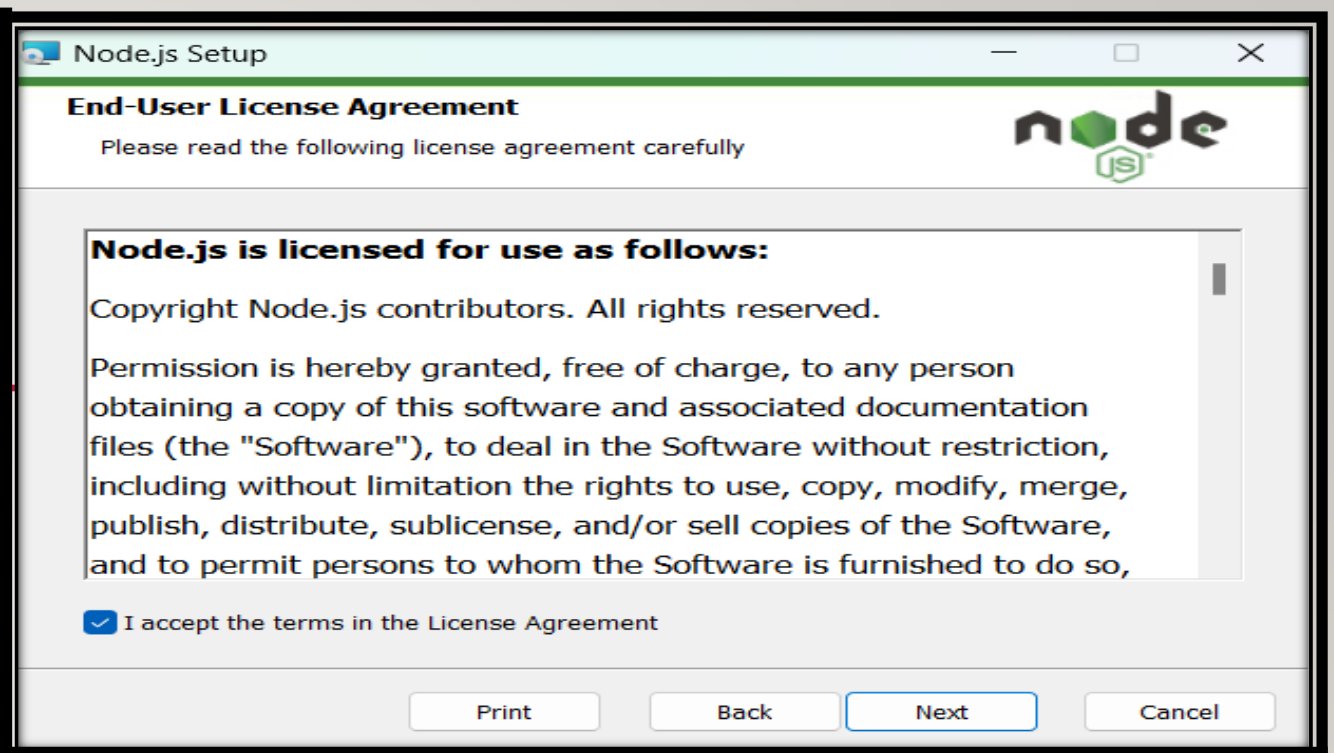
Download the Node.js source code or a pre-built installer for your platform, and start developing today.

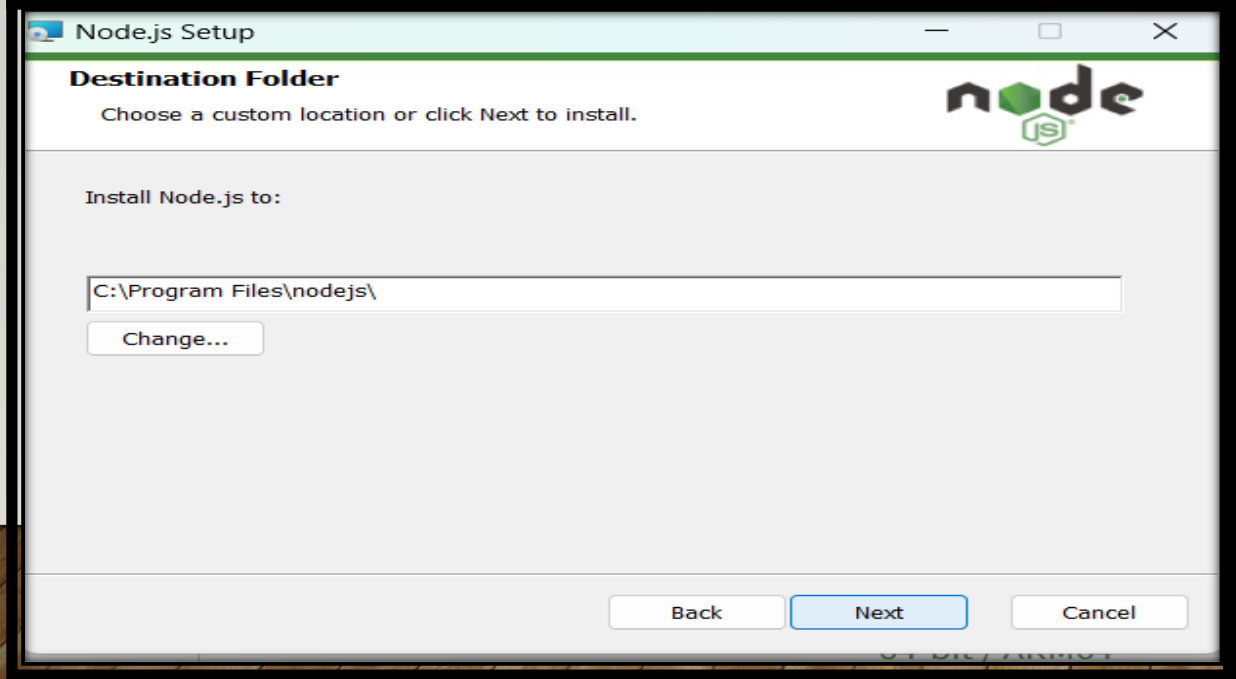
LTS Recommended For Most Users	Current Latest Features

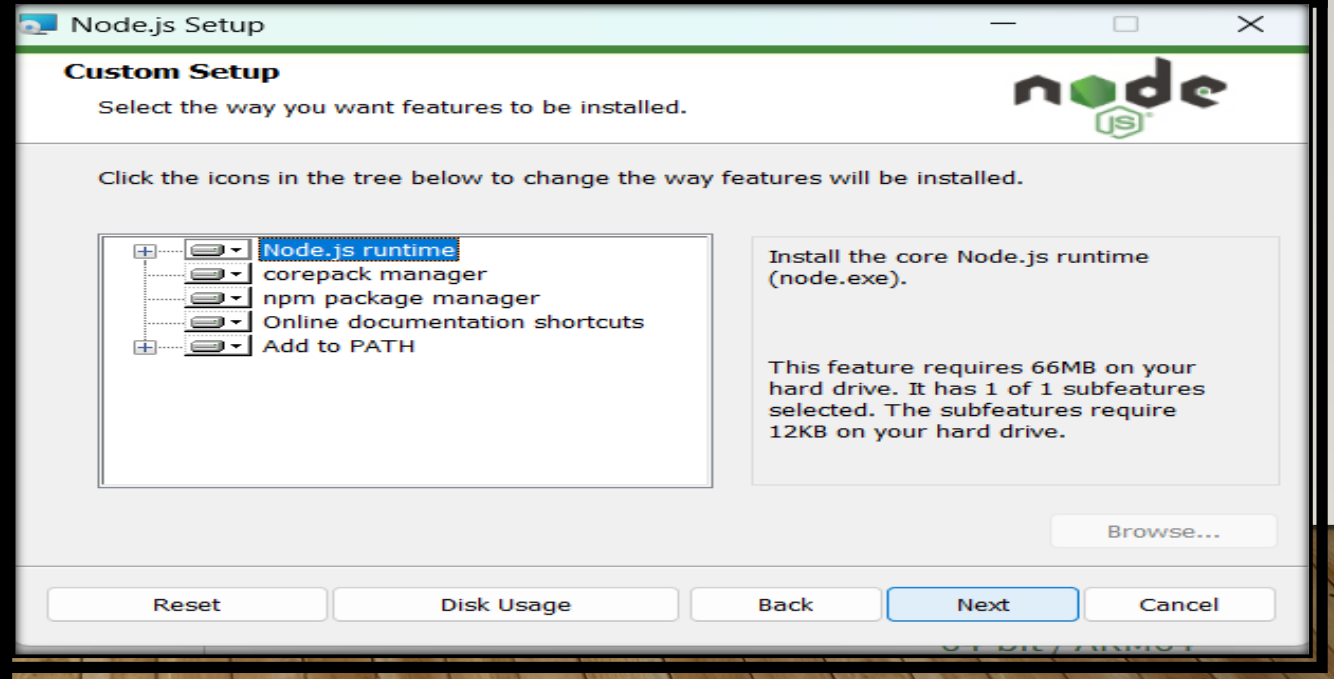
Windows Installer	macOS Installer	Source Code
node-v18.16.0-x64.msi	node-v18.16.0.pkg	node-v18.16.0.tar.gz

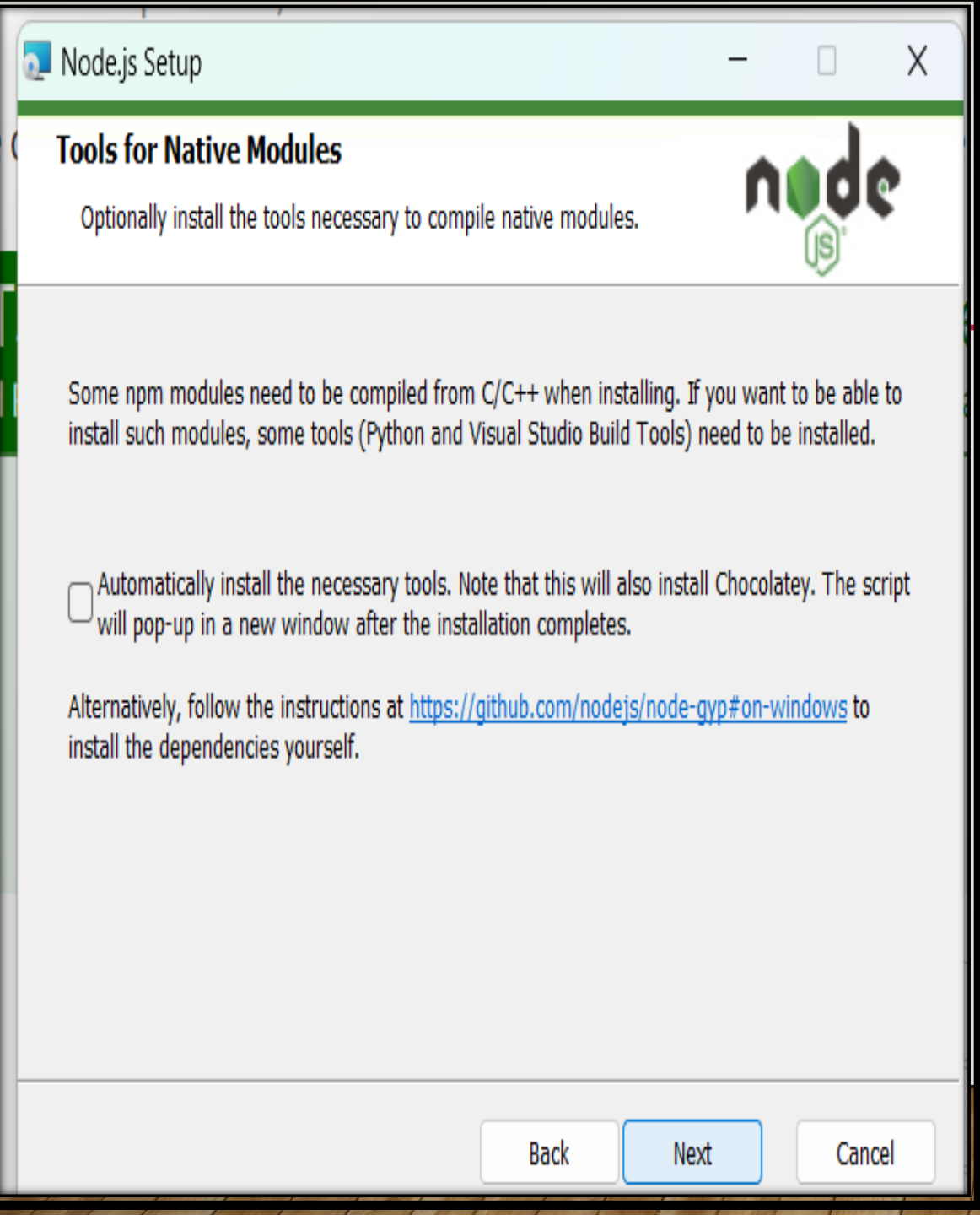
Windows Installer (.msi)	32-bit	64-bit
Windows Binary (.zip)	32-bit	64-bit
macOS Installer (.pkg)	64-bit / ARM64	
macOS Binary (.tar.gz)	64-bit	ARM64
Linux Binaries (x64)	64-bit	
Linux Binaries (ARM)	ARMv7	ARMv8
Source Code	node-v18.16.0.tar.gz	


Node.js Setup
Welcome to the Node.js Setup Wizard
The Setup Wizard will install Node.js on your computer.
Back
Next
Cancel

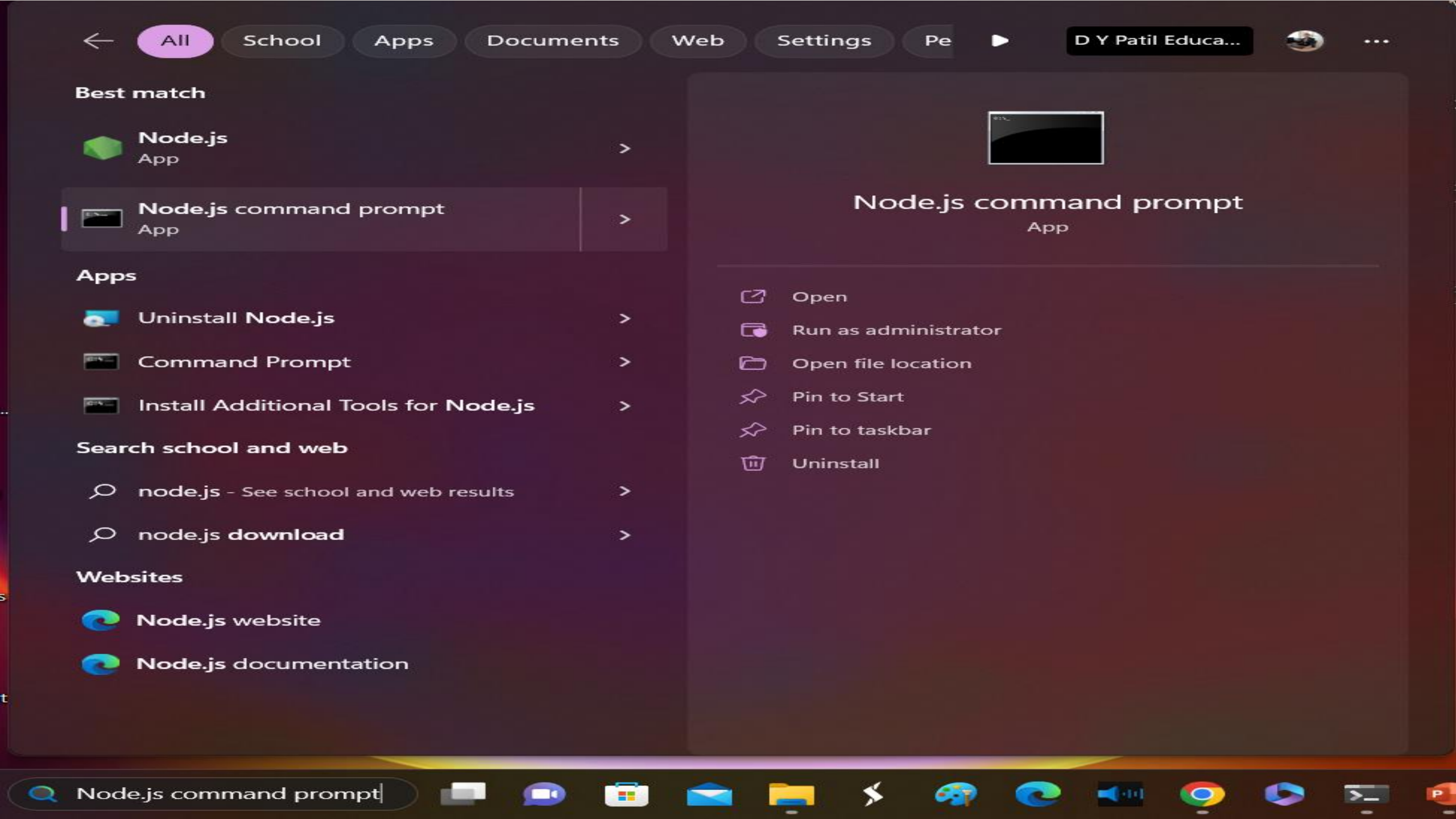

Node.js Setup
End-User License Agreement
Please read the following license agreement carefully
Node.js is licensed for use as follows:
Copyright Node.js contributors. All rights reserved.
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so,
I accept the terms in the License Agreement
Print
Back
Next
Cancel


Node.js Setup
Destination Folder
Choose a custom location or click Next to install.
Install Node.js to:
C:\Program Files\nodejs\
Change...
Back
Next
Cancel


Node.js Setup
Custom Setup
Select the way you want features to be installed.
Click the icons in the tree below to change the way features will be installed.
Node.js runtime
corepack manager
npm package manager
Online documentation shortcuts
Add to PATH
Install the core Node.js runtime (node.exe).
This feature requires 66MB on your hard drive. It has 1 of 1 subfeatures selected. The subfeatures require 12KB on your hard drive.
Browse...
Reset
Disk Usage
Back
Next
Cancel

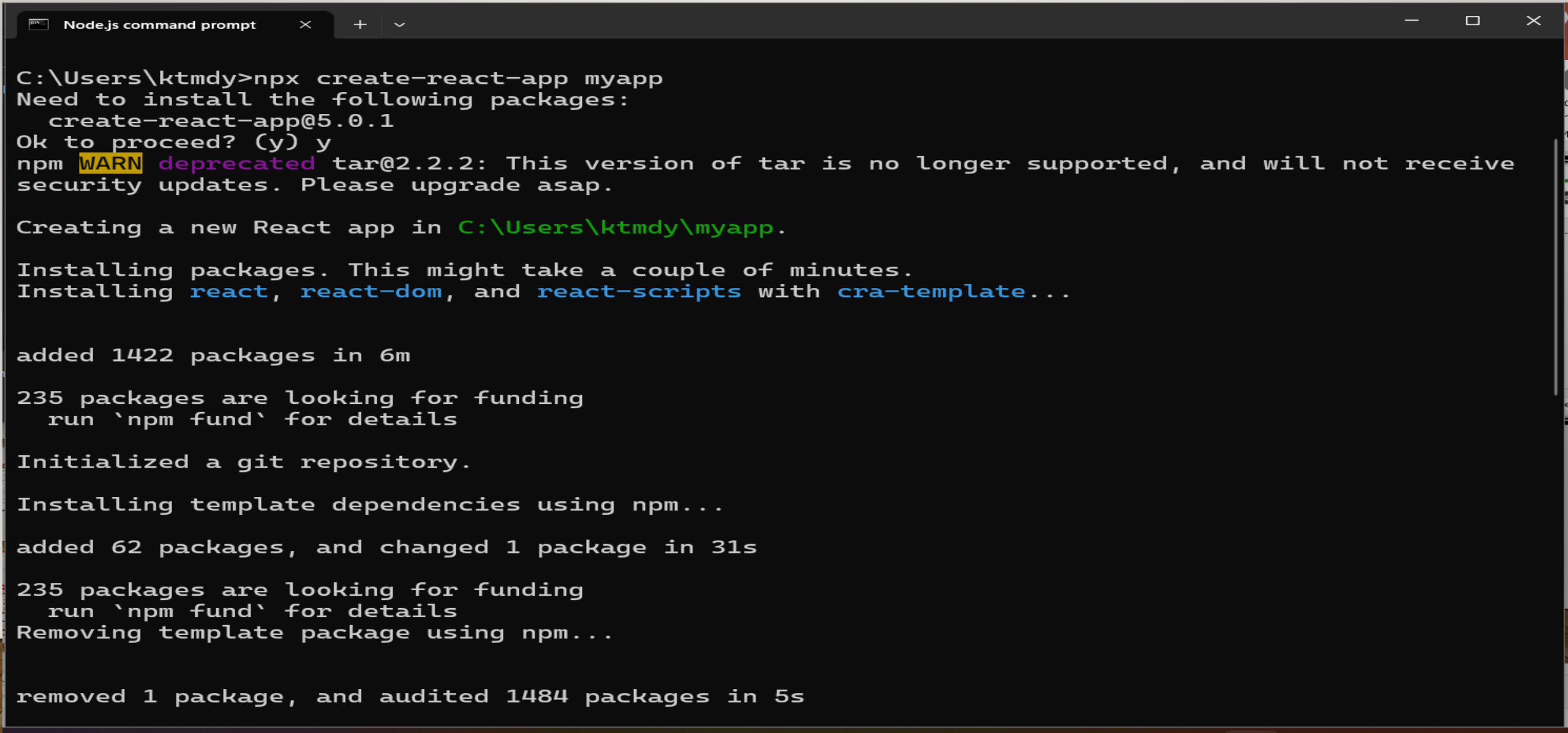

Node.js Setup
Tools for Native Modules
Optionally install the tools necessary to compile native modules.
Some npm modules need to be compiled from C/C++ when installing. If you want to be able to install such modules, some tools (Python and Visual Studio Build Tools) need to be installed.
Automatically install the necessary tools. Note that this will also install Chocolatey. The script will pop-up in a new window after the installation completes.
Alternatively, follow the instructions at https://github.com/nodejs/node-gyp#on-windows to install the dependencies yourself.
Back
Next
Cancel


Node.js Setup
Completed the Node.js Setup Wizard
Click the Finish button to exit the Setup Wizard.
Node.js has been successfully installed.
Back
Finish
Cancel


All
School
Apps
Documents
Web
Settings
Pe
D Y Patil Educa...
Best match
Node.js
App
Node.js command prompt
App
Apps
Uninstall Node.js
Command Prompt
Install Additional Tools for Node.js
Search school and web
node.js - See school and web results
node.js download
Websites
Node.js website
Node.js documentation
Node.js command prompt
App
Open
Run as administrator
Open file location
Pin to Start
Pin to taskbar
Uninstall
Node.js command prompt

TO INSTALL REACTJS AND CREATE APP GIVE COMMAND IN NODE.JS COMMAND PROMPT

npx create-react-app myapp



Node.js command prompt

```
To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

Created git commit.

Success! Created myapp at C:\Users\ktmdy\myapp
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

  cd myapp
  npm start

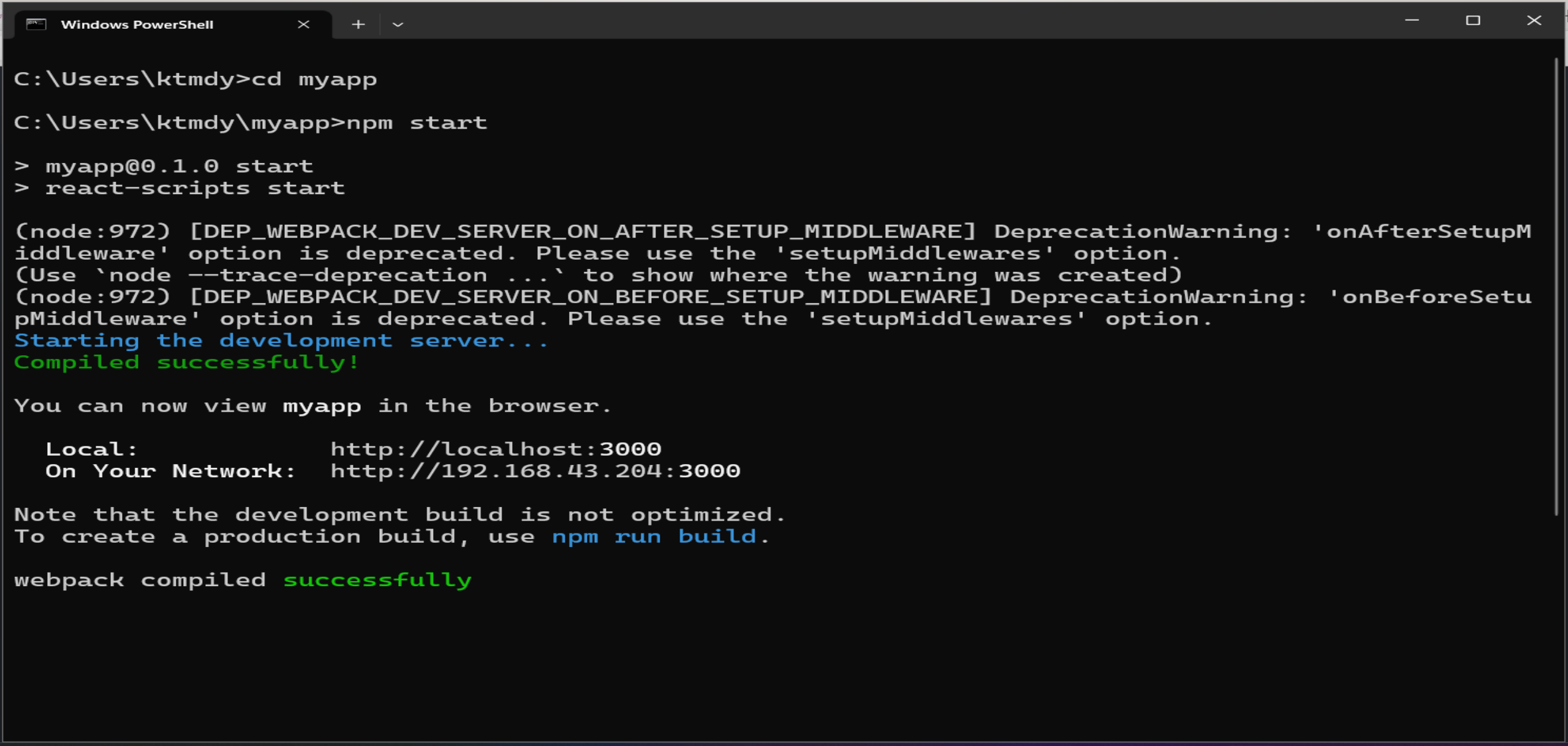
Happy hacking!

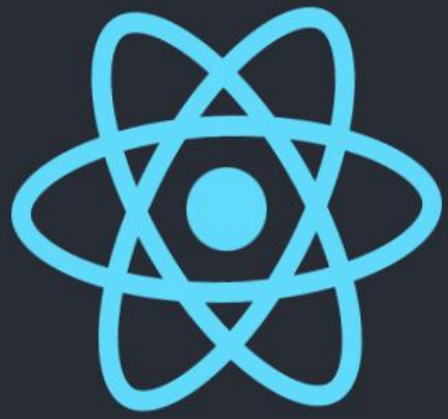
C:\Users\ktmdy>
```

It will install the reactJS library and create the application with name myapp

Search

TO START THE APPLICATION SERVER DO FOLLOWING COMMANDS



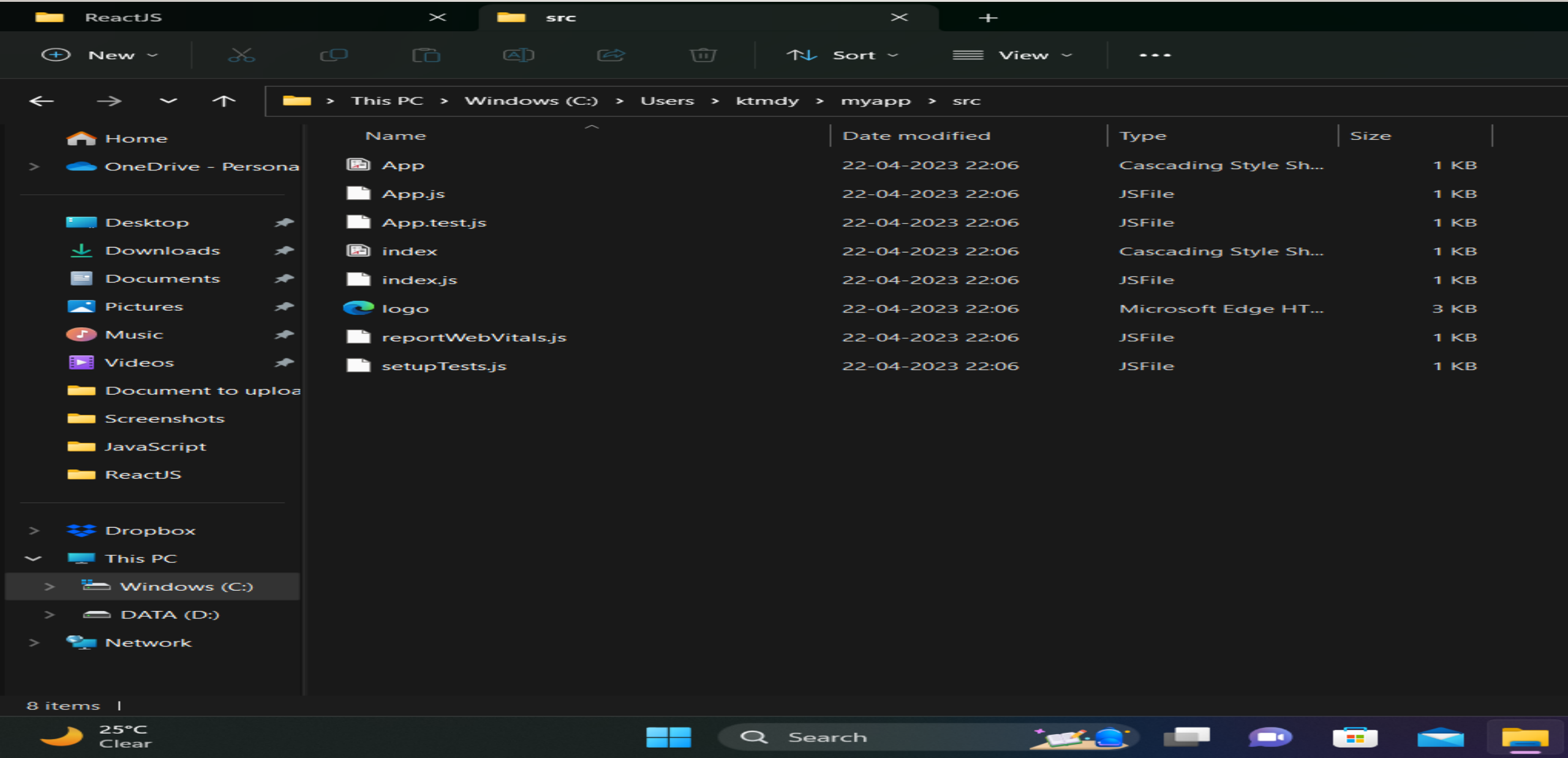


Edit `src/App.js` and save to reload.

Learn React

Application creates the source files in C:\Users\ktmdy\myapp\

Src folder includes following files used to write code



To write our own code add code to app.js file

```
import React from "react";

class HelloWorld extends React.Component

{

render() {

   return (

          <div>

                <h1>Hello World!</h1>

          </div>

            );

          }

}export default HelloWorld;
```

COMPONENTS

- React component is the building block of a React application
- Two types –

1. Function Component
2. Class component

CLASS COMPONENT

- The class components are a little more complex than the functional components.
- The functional components are not aware of the other components in your program whereas the class components can work with each other.
- We can pass data from one class component to other class components.
- We can use JavaScript classes to create class-based components in React. Below example shows a valid class-based component in React:

```
class Democomponent extends React.Component
{
   render(){
                    return <h1>Welcome Message!</h1>;
            }
 }
```

FUNCTIONAL COMPONENT

- Functional components are simply javascript functions.
- We can create a functional component in React by writing a javascript function.

Functional Components	Class Components
A functional component is just a plain JavaScript pure function that accepts props as an argument and returns a React element(JSX).	A class component requires you to extend from React. Component and create a render function which returns a React element.
There is no render method used in functional components.	It must have the render() method returning JSX –Java Script XML (which is syntactically similar to HTML)
Functional component run from top to bottom and once the function is returned it cant be kept alive.	Class component is instantiated and different life cycle method is kept alive and being run and invoked depending on phase of class component.
Also known as Stateless components as they simply accept data and display them in some form, that they are mainly responsible for rendering UI.	Also known as Stateful components because they implement logic and state.
React lifecycle methods (for example, componentDidMount) cannot be used in functional components.	React lifecycle methods can be used inside class components (for example, componentDidMount).

Hooks can be easily used in functional components to make them Stateful.
example: const [name,SetName]= React.useState(‘ ‘)
	It requires different syntax inside a class component to implement hooks.
	example: constructor(props) {
	super(props);
	this.state = {name:‘ ‘}
	}

Constructors are not used.	Constructor are used as it needs to store state.

To write our own code add code to app.js file

CLASS COMPONENT

```
import React from "react";

class HelloWorld extends React.Component

{

render() {

   return (

          <div>

                <h1>Hello World!</h1>

          </div>

            );

          }

}export default HelloWorld;
```

To write our own code add code to app.js file inside src folder

FUNCTION COMPONENT

```
import React from "react";
    function Hello()
     {
          return (
                    <div>
                         <h1>Hello World!</h1>
                    </div>
                      );
  }export default Hello;
```

STYLING

- React allows component to be styled using CSS class through className attribute.
- Three important methodology to style our component -

1. CSS Stylesheet
2. Inline Styling
3. CSS Modules

1. CSS STYLESHEET

Save file in src folder

- Test1.css

```
div {
    color: red;
    font-size: 14px;
}
```

Save code in app.js file

```
import React from "react";
import './test1.css';
class HelloWorld extends React.Component {
  render() {
        return (
      <div>
          <h1>Hello World!</h1>
      </div>
            );
  }}export default HelloWorld;
```

2. INLINE STYLING

Save code in app.js file

```
import React from "react";
  class HelloWorld extends React.Component {
      render() {
                  return (
                        <div style={ {
                                    color: 'red',    fontSize: '14px'
                              }}>
                            <h1>Hello World!</h1>
                        </div>
                      );
  }}export default HelloWorld;
```

3. CSS MODULES

- It uses normal css stylesheet with normal syntax.
- While importing the styles, CSS modules converts all the styles into locally scoped styles so that the name conflicts will not happen.
- Let us change our component to use *CSS modules*

CSS MODULE

Save file in src folder

- Test1.module.css

```
div {
    color: red;
    font-size: 14px;
}
```

Save code in app.js file

```
import React from "react";
import styles from './test1.module.css'
class HelloWorld extends React.Component {
  render() {
        return (
            <div className={styles} >
              <h1>Hello World!</h1>
            </div>
          );
  }}export default HelloWorld;
```